Design Note: Low-Level Design (LLD) Practice Platform

Author: Govind Kumar **Date:** October 2026 **Assignment:** 2-Day Engineering Prototype

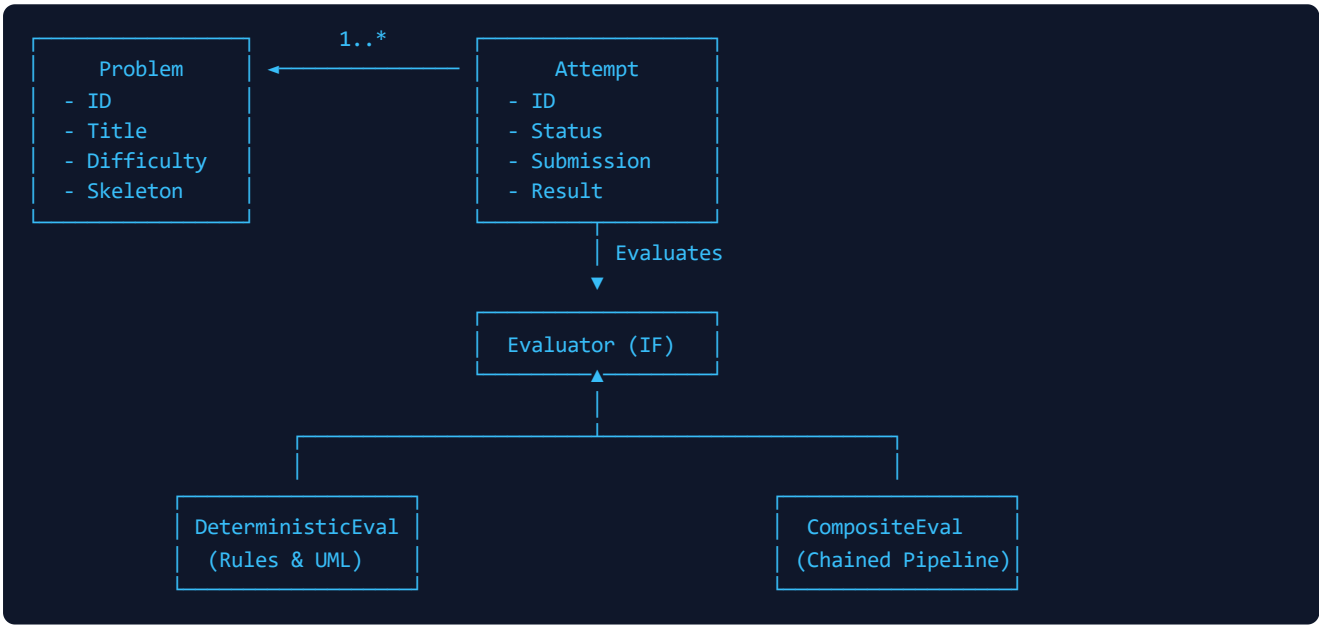
1. System Overview & Practice Loop

The platform is designed around the core learner iteration cycle, prioritizing **explainable, evidence-backed architectural feedback** and **rapid re-attempt learning**:

Choose Problem → Design Code & Notes → Submit → Rubric Scorecard → Inspect Dynamic UML → Iterate Attempt

2. Core Domain Model & Decoupled Architecture

The domain model cleanly separates problem specification, candidate submission, persistence lifecycle, and evaluation pipelines:



Key Domain Entities:

- **`Problem`:** Encapsulates scenario metadata, difficulty, description, and initial class skeleton.
- **`Submission`:** Encapsulates candidate payload with format abstraction (`FormatCodeAndNotes`, `FormatClassDiagram`, `FormatSpecification`).
- **`Attempt`:** Models practice iterations, owning an explicit state lifecycle: `SUBMITTED → EVALUATING → COMPLETED / FAILED`.
- **`Evaluator` (Interface):** Core architectural contract decoupling evaluation logic from HTTP and storage layers.
- **`RubricDimension`:** 5-point structured feedback unit: `Criterion → Score → Evidence → Concern → Suggestion → Confidence`.

3. Addressing the Assignment's Two Critical "Change Tests"

Change Test A: Changing Submission Format

"Today the learner submits text/code. Later the platform supports a class diagram. How much of your domain model changes?"

Design Solution: The domain introduces `SubmissionFormat` as an explicit enum abstraction. The `Submission` entity defines:

```
type Submission struct {
    ProblemID   int              `json:"problem_id"`
    Format      SubmissionFormat `json:"format,omitempty"`
    CodeContent string           `json:"code_content"`
    DesignNotes string           `json:"design_notes"`
    DiagramData string           `json:"diagram_data,omitempty"`
}
```

Impact: Adding visual class diagrams or specifications requires **zero changes** to `Problem`, `Attempt`, or database tables. A new diagram parser strategy simply evaluates `DiagramData`.

Change Test B: Adding New Evaluation Strategies

"Today feedback comes from one evaluator. Later you add a rule-based evaluator, human review, or an LLM. Can you add it without rewriting the practice flow?"

Design Solution: The Open/Closed Principle is applied via the `Evaluator` interface and Composite Pattern:

```
type Evaluator interface {
    Name() string
    Evaluate(ctx context.Context, sub Submission, prob Problem) (EvaluationResult, error)
}
```

Impact: Chaining an LLM or AST evaluator only requires implementing `Evaluator` and calling `composite.AddEvaluator(newEval)`. The HTTP handlers and persistence logic remain completely untouched.

4. Answers to Core Design Questions (Section 4)

- **Q1: What does a learner provide?** Classes/interfaces (demonstrating encapsulation and hierarchy) + architectural notes (justifying trade-offs, concurrency handling, and pattern selection).
- **Q2: What makes feedback useful when multiple valid solutions exist?** Evaluating across multi-dimensional criteria (SRP, DIP/ISP, Extensibility, Concurrency) and citing concrete code evidence rather than comparing against a rigid template.
- **Q3: Deterministic vs. LLM benefits:** Deterministic rules excel at syntax validity, domain mismatch rejection, and dynamic UML parsing. LLMs excel at nuanced semantic review and qualitative trade-off critique.
- **Q4: Handling slow/failing evaluations:** Submissions are persisted in SQLite with status `EVALUATING` before processing begins, guarded by a 10-second Go `context.WithTimeout`, preventing candidate data loss on pipeline timeouts.

5. Key Engineering Trade-offs

Decision	Chosen Approach	Alternative Considered	Engineering Rationale
Architecture	Go Monolith (Fiber)	Microservices (API + Worker)	Minimizes operational complexity for MVP; sub-millisecond local execution without distributed failure points.
Storage	SQLite (WAL Mode)	PostgreSQL / MongoDB	Zero-configuration local persistence with ACID guarantees and complete portability.
UML Rendering	Dynamic client-side Mermaid.js	Server-side Graphviz/PlantUML	Parses AST tokens directly from code into interactive SVG diagrams without external binary dependencies.